TrustFence

Intent-Bound AI Action Firewall

An evidence-first policy gate that stops untrusted emails, documents and tickets from expanding an AI agent's authority.

THEME	PRIMARY RISK	CORE CONTROL
AI & Cybersecurity	Indirect prompt injection	Intent-to-action policy gate

Executive summary

AI agents increasingly consume emails, tickets, webpages and documents while operating near privileged tools. Those sources are data, not trusted administrators, yet they may contain malicious instructions asking an agent to export records, forward sensitive files, delete evidence or change access. Detecting suspicious text is useful, but the operational question is whether the requested action is within the scope the user actually authorized.

TrustFence creates a narrow "intent contract" from the original trusted request, analyzes untrusted content for instruction overrides and risky actions, and compares the two. Out-of-scope side effects are blocked with a visible evidence trace. Ambiguous instruction attempts are held for human review, and even actions apparently requested by the user require a separate approval flow. The hackathon MVP demonstrates this policy with synthetic examples, a local browser dashboard, a deterministic rule engine, and automated tests. It does not connect to real accounts or execute actions.

The principle is to separate understanding from authorization: an AI model may help interpret content, but the model itself must not grant permissions. Authorization stays in a deterministic, server-enforced policy gate, with least-privilege tool scopes and human confirmation for privileged actions.

1. Problem and threat model

A common failure mode occurs when a trusted assistant reads an untrusted source and treats the source's instructions as commands. An attacker may hide or embed a directive such as "ignore previous instructions", then request a side effect through a tool the agent can access. The threat is amplified by excessive functionality, permissions or autonomy. The prototype focuses on this concrete chain rather than trying to solve all phishing or endpoint-security problems.

- Assets at risk: confidential files, credentials, email accounts, audit records, access controls and trusted workflow decisions.
- Trust boundary: original user request is the source of task authority; email/document/ticket content is untrusted evidence.
- Covered attack classes: instruction override, secret disclosure, external data transfer, deletion, command execution and permission changes.
- Out of scope for this MVP: live integrations, malware analysis, model training, enterprise identity, and claims of comprehensive prevention.

2. Proposed solution

01 · Intent contract	Extract the narrow purpose and allowed action types from the original user request. If the task is “summarize”, remain read-only.
02 · Untrusted-content scan	Identify instruction overrides, concealment, secret access, data movement, destructive actions and privilege changes.
03 · Scope-drift diff	Show each proposed action next to its source evidence and whether that action is covered by the trusted user request.
04 · Enforceable decision	Block out-of-scope side effects; hold ambiguous injection for review; require confirmation for legitimate side effects.
05 · Audit trace	Record decision, reason, evidence excerpt and next step for later review. Sensitive evidence should be redacted in production.

3. What makes this a stronger hackathon direction

Do not claim this is the first prompt-injection detector or that no one has ever proposed human approval. Existing AI security assurance efforts already cover related risks. TrustFence’s specific product thesis is a low-friction, evidence-first guardrail around existing agent workflows that compares the authority of the original user request with action instructions found in untrusted content. The demo visualizes “scope drift” directly, helping a reviewer reason about what was requested and why the policy blocks it.

- A concrete before/after demo: benign read-only document vs. injected export/email/delete instructions.
- Explainability is in the core workflow: each finding includes the matched evidence, not just a red/green verdict.
- Safety-first design: the model can propose an action, but cannot authorize it; action execution is outside the current demo.
- Measurable hypothesis: an intent-bound gate can block out-of-scope side effects while preserving benign read-only tasks.

4. MVP scope and system architecture

Browser UI → original-user-intent field + untrusted-content field → evidence extractor → intent contract → deterministic policy gate → decision card and audit-style trace. The prototype is built with Node.js built-ins and a small HTML/CSS/JavaScript frontend so a judge can run it locally without a database or paid API key.

Included in the starter bundle

- Local demo dashboard and scenario buttons.
- Rule engine for five instruction signals and six risky-action categories.
- Eight initial synthetic scenarios and eight automated tests.
- README with run instructions, demo script, architecture and security limitations.
- Research and form-copy notes for the idea-submission phase.
- Optional local Ollama semantic adapter; policy authorization remains deterministic.

Future version after the idea phase

- Expand semantic extraction and paraphrase coverage; model output remains an untrusted proposal.
- Policy-enforcement adapter at the actual tool boundary, with identity-bound action scopes and least privilege.
- Human approval card showing recipient, destination, data class, impact and rollback plan.
- Replayable red-team benchmark with benign/adversarial pairs, signed evidence logs and framework mapping.

5. Evaluation plan

The bundled examples are synthetic engineering fixtures, not a validated benchmark. Before the offline round, expand to at least 40 labeled examples across benign content, direct/indirect injection, secret disclosure, external transfer, deletion, privilege changes, and ambiguous text. Record results and error cases rather than claiming performance in advance.

Metric	How to measure	Why it matters
Unauthorized actions blocked	Blocked out-of-scope proposals / all attempted out-of-scope proposals	Measures policy enforcement on the labeled set.
False-block rate	Benign cases incorrectly blocked / all benign cases	Ensures security does not make the workflow unusable.
Evidence-review time	Time for reviewers to find and explain the trigger	Tests the value of the evidence-first UI.
Read-only task completion	Benign summaries completed without broad permission	Shows utility, not just detection.
Latency	Median and p95 local analysis time	Supports deployment feasibility.

6. Build plan before the offline hackathon

1. Day 1: confirm team and requirements, run the demo, refine the title and problem framing, review official submission options.
2. Day 2: add labeled benign/adversarial fixtures, run tests, record a 90-second demo, and revise the deck with measured evidence only.
3. Before 31 Oct: implement optional semantic layer and improve paraphrase coverage; add policy API boundaries and a deployable demo if shortlisted.
4. At the offline round: show a two-case contrast first, then open the evidence and policy trace; explain limits candidly and show the benchmark.

7. Risk, ethics and limitations

- Detection is incomplete: regexes miss paraphrases, multilingual attacks, images and obfuscated content; false positives are possible.
- A frontend verdict is not a security boundary. Production enforcement must occur at the real tool/API call, server-side.
- The demo uses synthetic text only. Do not paste confidential customer data, API keys or real incident records into a public demo.
- Do not report the initial eight examples as accuracy or benchmark results. Expand the corpus and show the confusion matrix.
- Never auto-execute email sending, deletion, access changes or export actions in a hackathon demonstration.

8. Submission copy

TrustFence is an evidence-first, intent-bound action firewall for AI agents. It protects workflows where agents read emails, tickets and documents by treating external content as untrusted data rather than an authority source. It derives a narrow action contract from the original user request, identifies instruction overrides and risky proposed actions, and compares each action against that contract. Out-of-scope side effects are blocked with the exact evidence shown to the reviewer; ambiguous attempts are held for review, and permitted side effects still require human confirmation. The MVP demonstrates this workflow using synthetic scenarios, an explainable policy engine, a browser dashboard and automated tests. Our design separates AI-assisted understanding from deterministic authorization, so a model can help interpret an action but cannot grant permissions. We will measure blocked unauthorized actions, false-block rate and evidence-review time on a labeled benign/adversarial set. The current prototype is not a production integration and makes no claim of perfect detection.

9. Research references

- OWASP GenAI Security Project, OWASP Top 10 for Large Language Model Applications (2026, published 4 Aug 2026). https://github.com/GenAI-Security-Project/GenAI-LLM-Top10
- OWASP GenAI Security Project, LLM01:2025 Prompt Injection. https://genai.owasp.org/llmrisk/llm01-prompt-injection/
- OWASP GenAI Security Project, LLM06:2025 Excessive Agency. https://genai.owasp.org/llmrisk/llm062025-excessive-agency/
- MITRE ATT&CK Enterprise, Indirect Prompt Injection (T1698). https://attack.mitre.org/techniques/T1698/
- MITRE ATT&CK Enterprise, AI-Enabled Exfiltration (T1700). https://attack.mitre.org/techniques/T1700/
- OWASP GenAI Security Project, Assurance-AI (adjacent work; published 4 Oct 2026). https://genai.owasp.org/solution/assurance-ai/

Prepared for idea-stage submission · 17 October 2026 · Team member names and travel commitment must be confirmed by the participants.